

Exercise 1: Control Structures

Scenario 1: Apply Loan Interest Rate Discount

Requirement: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

```
DECLARE
    -- Cursor to fetch customer records along with their active loans
    CURSOR c_senior_loans IS
        SELECT c.CustomerID, c.DOB, l.LoanID, l.InterestRate
        FROM Customers c
        JOIN Loans l ON c.CustomerID = l.CustomerID;

    v_age NUMBER;
BEGIN
    FOR r_loan IN c_senior_loans LOOP
        -- Calculate exact age based on Date of Birth
        v_age := MONTHS_BETWEEN(SYSDATE, r_loan.DOB) / 12;

        -- If customer is over 60, update the loan interest rate
        IF v_age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE LoanID = r_loan.LoanID;

            DBMS_OUTPUT.PUT_LINE('Discount applied for Customer ID: ' ||
r_loan.CustomerID);
        END IF;
    END LOOP;

    COMMIT;
END;
/
```

Scenario 2: Promote Customers to VIP Status

Requirement: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

Schema Modification Note: Since the core Customers table structure does not natively include the IsVIP flag, make sure to execute the structural update prior to using the block:

```
ALTER TABLE Customers ADD IsVIP VARCHAR2(5);
```

```
DECLARE
    CURSOR c_customers IS
        SELECT CustomerID, Balance
        FROM Customers;
BEGIN
    FOR r_cust IN c_customers LOOP
        -- Check if customer balance exceeds the $10,000 baseline threshold
        IF r_cust.Balance > 10000 THEN
            UPDATE Customers
            SET IsVIP = 'TRUE'
            WHERE CustomerID = r_cust.CustomerID;

            DBMS_OUTPUT.PUT_LINE('Customer ID ' || r_cust.CustomerID ||
' promoted to VIP.');
        END IF;
    END LOOP;

    COMMIT;
END;
/
```

Scenario 3: Loan Due Reminders

Requirement: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
DECLARE
    CURSOR c_due_loans IS
        SELECT l.LoanID, c.Name, l.EndDate, l.LoanAmount
        FROM Loans l
        JOIN Customers c ON l.CustomerID = c.CustomerID
        WHERE l.EndDate BETWEEN SYSDATE AND (SYSDATE + 30);
```

```

BEGIN
    FOR r_loan IN c_due_loans LOOP
        DBMS_OUTPUT.PUT_LINE('Reminder: Loan ID ' || r_loan.LoanID ||
                               ' for ' || r_loan.Name ||
                               ' of amount $' || r_loan.LoanAmount ||
                               ' is due on ' || TO_CHAR(r_loan.EndDate, 'YYYY-MM-DD')
                               || '.');
    END LOOP;
END;
/

```

Exercise 3: Stored Procedures

Scenario 1: Process Monthly Interest

Requirement: Write a stored procedure `ProcessMonthlyInterest` that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```

CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS
BEGIN
    -- Update target accounts meeting specific account criteria
    UPDATE Accounts
    SET Balance = Balance * 1.01,
        LastModified = SYSDATE
    WHERE AccountType = 'Savings';

    DBMS_OUTPUT.PUT_LINE('Monthly interest of 1% applied to all Savings accounts.');
```

COMMIT;

```

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error processing monthly interest: ' || SQLERRM);
END ProcessMonthlyInterest;
/

```

Scenario 2: Employee Bonus Scheme

Requirement: Write a stored procedure `UpdateEmployeeBonus` that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
    p_department IN Employees.Department%TYPE,
    p_bonus_pct  IN NUMBER
) IS
BEGIN
    -- Apply incremental percentage adjustments globally to the parameter matching
    unit
    UPDATE Employees
    SET Salary = Salary + (Salary * (p_bonus_pct / 100))
    WHERE Department = p_department;

    DBMS_OUTPUT.PUT_LINE('Bonus applied to department: ' || p_department);
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error updating employee bonus: ' || SQLERRM);
END UpdateEmployeeBonus;
/

```

Scenario 3: Fund Transfer Procedure

Requirement: Write a stored procedure `TransferFunds` that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```

CREATE OR REPLACE PROCEDURE TransferFunds (
    p_source_acc IN Accounts.AccountID%TYPE,
    p_dest_acc   IN Accounts.AccountID%TYPE,
    p_amount     IN NUMBER
) IS
    v_source_balance NUMBER;
BEGIN
    -- Fetch current balance of the source account
    SELECT Balance INTO v_source_balance
    FROM Accounts
    WHERE AccountID = p_source_acc;

    -- Assert transactional logic integrity constraints
    IF v_source_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient balance in source account.');
```

ELSE

```

        -- Perform multi-step synchronous atomic adjustments
        UPDATE Accounts
        SET Balance = Balance - p_amount,

```

```

        LastModified = SYSDATE
    WHERE AccountID = p_source_acc;

    UPDATE Accounts
    SET Balance = Balance + p_amount,
        LastModified = SYSDATE
    WHERE AccountID = p_dest_acc;

    DBMS_OUTPUT.PUT_LINE('Successfully transferred $' || p_amount ||
                          ' from Account ' || p_source_acc || ' to Account ' ||
p_dest_acc);
    COMMIT;
END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: One or both account IDs do not exist.');
```

```

    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Transaction rolled back due to error: ' || SQLERRM);
END TransferFunds;
/
```